# Laboratory 5
# Transformers

## Objectives

In this laboratory, you will study Vision Transformers. Unlike CNNs, which rely on convolution and inductive biases such as locality and translation equivariance, Vision Transformers model global relationships using self-attention. The goal of this lab is not only to fine-tune a Transformer-based model, but to understand how attention works in images and how positional encodings work.

## Proposed problems

**Attention implementation**

Self-attention is the operation that replaces convolution in a Vision Transformer. Instead of processing information locally with a convolutional kernel, it mixes information globally by letting every token "look at" every other token and decide how much it matters. In the context of computer vision, the input to attention is a sequence of image tokens. After patchifying an image and projecting patches to an embedding space, you obtain a tensor $X \in R^{B \times T \times D}$, where B is the batch size, T is the number of tokens (patch tokens and, optionally, the CLS token), and D is the embedding dimension**.** First, attention constructs for each token, three learned projections: queries, keys, and values by using three linear layers:

$$Q = XW_Q ; K = XW_K ; V = XW_V$$

, with $W_Q$ , $W_K$, $W_V \in R^{D \times D}$. The results $Q$ ,K, $V \in R^{B \times T \times D}$
Attention scores are computed as token-to-token similarities between queries and keys, scaled for numerical stability:

$$S = \frac{QK^T}{\sqrt{D}}$$

, where $K^T$ is the transpose operation over the last two dimensions, so $S \in R^{B \times T \times T}$

You then apply softmax over the last dimension (over "which tokens I attend to") to compute attention weights:

$$A = softmax(S).$$

Finally to obtain the output Y, you mix the computed values V using these weights:

$$Y = AV,$$

where $Y \in R^{B \times T \times D}$.

This is single-head self-attention: each token outputs a weighted average of value vectors from all tokens, with weights decided by query–key similarity.

The intuition Andrew Karpathy often uses is that self-attention behaves like a message passing in a fully connected graph. Each token is a node in this graph and every node is connected to every other node.
Remember that three vectors are computed for each token: a query, a key, and a value. The query represents what kind of information the token is looking for. The key represents what kind of information the token contains. The value is the actual content that can be shared.
In one attention layer, every token can aggregate information from the entire image in a single step. When multiple layers are stacked, this process repeats. Tokens exchange and refine information, gradually forming more global and semantically meaningful representations.

To implement

- Create a torch.nn.Module called SelfAttention that takes embed_dim=D as an argument.
- In __init__, define three linear projections Wq, Wk, Wv (each as nn.Linear(D, D, bias=False)), and optionally an output projection Wo = nn.Linear(D, D, bias=False) if you want the classic formulation.
- In forward, accept an input x with shape (B, T, D).
- Compute q, k, v with shapes (B, T, D).
- Compute attention scores with a batched matrix multiply so that the result has shape (B, T, T), and scale by sqrt(D).
- Apply softmax over the last dimension to obtain attention weights of shape (B, T, T).
- Multiply the weights by v to obtain the output (B, T, D), and return it (apply Wo if you included it).
- For debugging and later visualization, also return the attention matrix A (or at least make it accessible).

Now, think on how you can modify this class to implement multi-headed self attention.

**Attention visualization**

Transformers are sometimes considered more interpretable than CNNs because they expose attention maps. Attention visualization allows you to study how information flows across tokens and how this behavior changes across depth. For this section, you will use a pretrained Vision Transformer from *torchvision*. **You do not need to train the model**! Load pretrained weights, put the model to evaluation mode, and analyze a few selected images.

*Single layer attention visualization*
First, extract attention weights from the transformer encoder blocks. You can do this by registering forward hooks on the multi-head self-attention modules. Each block produces attention tensors of shape $heads \times token \times token$, where tokens correspond to image patches plus the class token.
A simple visualization is the attention from the CLS token to the patch tokens:

a. Select the attention row corresponding to the CLS token.
b. Remove the CLS-to-CLS entry.
c. Reshape the remaining values into the patch grid.

d. Upsample to image resolution.
e. Overlay the attention map as a heatmap on the original image.

Do this for different layers in the model and also for individual heads and for averaged attention heads. Check how early vs deeper layers attend to the image. Are some heads nearly uniform or noisy (possible redundancy)?

*Attention Rollout*

Single-layer attention maps do not show the full information flow through the architecture. Attention rollout is a simple technique that composes attention matrices across layers to estimate how input patches influence the final CLS representation. The steps are the following:

1. For each transformer layer $l$, obtain the attention matrix $A_l$ (dimensions: $token \times token$) after softmax normalization.
2. Because residual connections add identity flow, many implementations add the identity matrix I to each attention matrix before composition: $A`_l = A_l + I$.
3. Normalize rows so that each row sums to 1. This keeps the matrix interpretable as a transition matrix that describes how information flows from one token to others.
4. Compose attention from the first layer to the last layer, using standard (Hadamard) matrix multiplication:
$$R = A`_L \times \dots \times A`_2 \times A`_1 .$$
5. The rollout matrix R shows aggregated attention flow; row or column sums can yield per-input importance scores.
6. To estimate which image patches influence the final classification:
    a. Take the row corresponding to the CLS token in R
    b. Remove the CLS-to-CLS entry.
    c. The remaining values represent the contribution of each patch to the final CLS representation.
    d. Reshape these values to the patch grid.
    e. Upsample and overlay as a heatmap on the original image.

Compare rollout maps with single-layer maps. Are they smoother? More global? More semantically meaningful?

**Implement and visualize positional encodings**

Transformers are permutation-invariant because self-attention treats tokens as a set, not a sequence. This means that once an image is divided into patches and converted into tokens, the model has no knowledge of where each patch comes from. Without additional information, swapping two patches would produce the same representation. Positional encodings solve this problem by injecting spatial structure into the embeddings.

In this section, you will implement **sinusoidal positional encodings** (proposed in the paper "*Attention is all you need*") from scratch.

For a sequence of length (T) and embedding dimension (d), the sinusoidal positional encoding is defined as:

$$PE(pos,\ 2i) = \sin \frac{pos}{10000^{\frac{2i}{d}}}$$

$$PE(pos,\ 2i + 1) = \cos \frac{pos}{10000^{\frac{2i}{d}}}$$

Here, *pos* is the token index (from 0 to *T*-1), and *i* indexes the embedding dimensions. Even dimensions use sine functions and odd dimensions use cosine functions. The denominator controls frequency: lower dimensions vary slowly across positions, while higher dimensions oscillate more rapidly.

In the original paper ("Attention is all you need"), the positional encoding matrix $PE \in R^{T \times d}$ is added directly to the token embeddings (X): $Z = X + PE$.

To better understand what you have implemented, visualize the positional encodings. Plot the full positional encoding matrix as a heatmap, with positions on one axis and embedding dimensions on the other. You should observe smooth, wave-like patterns that change frequency across dimensions. Optionally, compute the cosine similarity between positional vectors at different positions and visualize the similarity matrix. This shows how positional encodings encode relative distance information.
Think on how frequency varies across dimensions, whether nearby positions appear more similar than distant ones, and how increasing embedding dimension changes the encoding.

**Fine-tuning a Vision Transformer model**

In this section, you will fine-tune a pretrained Vision Transformer on CIFAR-10 (it's a small dataset we propose for pedagogical purposes; if you have powerful GPUs, you can choose a different dataset).
Load a pretrained vision transformer model from torchvision (for example *vit_b_16* with pretrained weights). Replace the classification head so that the final linear layer outputs 10 classes. Make sure the input resolution matches what the model expects (resize CIFAR-10 images accordingly).
Start with linear probing. Freeze all transformer parameters and train only the final classification head. This means setting requires_grad=False for the backbone and keeping it True only for the head. Train for a few epochs and report training and validation accuracy.

Then perform full fine-tuning. Unfreeze the entire model and train all parameters. Use a smaller learning rate than in linear probing, since now you are updating the whole network. Compare results with the previous experiment. Does full fine-tuning improve accuracy? Does it overfit? How does convergence speed differ?
Comment on the difference between the two approaches. When does freezing the backbone make sense? When does full fine-tuning help?

~~**Parameter Efficient Fine Tuning**~~

~~Full fine-tuning updates all weights in the transformer, which can be expensive and causes several training problems. In this section, you will implement a parameter-efficient technique using LoRA (Low-Rank Adaptation). In self-attention, linear projections for queries, keys, and values are computed. Normally, these projections use weight matrices that are fully trainable. In LoRA, the original pretrained weights are frozen,~~

and instead of modifying them directly, a small low-rank update is added. In other words, the projection matrix W is replaced with W + ΔW, where ΔW is decomposed as the product of **two smaller matrices with low rank**: ΔW=BA, where $A \in R^{r \times D}$ and $B \in R^{D \times r}$.

The rank in LoRA is the dimension of the low-rank update applied to a weight matrix. Instead of learning a full matrix of size $D \times D$, the update is constrained to have rank $r$, where $r \ll D$. It controls how many additional parameters are introduced and how expressive the update can be.

During training, only these two matrices ($A$ and $B$) will be fine-tuned, while keeping the original weight matrix $W$ frozen. Modify the attention projections (for example only the query and value projections) by adding LoRA adapters. Keep the original ViT weights frozen and train only the low-rank matrices and the classification head.

Compare the number of trainable parameters, training speed and final accuracy. Does LoRA approach the performance of full fine-tuning? How many parameters are you actually updating compared to the full model?

**To sum up, your tasks for this lab are:**

- Implement single-head self-attention from scratch as a torch.nn.Module.
- Verify your implementation by comparing it numerically with torch.nn.MultiheadAttention using one head.
- Extend your implementation to multi-head self-attention.
- Load a pretrained Vision Transformer from torchvision and extract attention maps using forward hooks.
- Visualize CLS-to-patch attention for different layers and heads, and compare early vs deeper layers.
- Implement attention rollout across layers and visualize the resulting heatmaps.
- Implement sinusoidal positional encodings from scratch and visualize them.
- Fine-tune a pretrained ViT on CIFAR-10 using linear probing (frozen backbone).
- Perform full fine-tuning of the entire model and compare results.
- Implement LoRA in the attention projections, freeze the backbone, and fine-tune only the low-rank adapters and the classification head.
- Compare linear probing, full fine-tuning, and LoRA in terms of trainable parameters, training time, and final accuracy.